

CS496 - Zero Knowledge Proof: From Theory to Application

Krishna Makwana
Guide: Prof. Adithya Vadapalli

November 13, 2025

Contents

Abstract	2
1 Theoretical Foundations	2
1.1 Definition: Conviction without Knowledge	2
1.2 The Three Core Properties	2
1.3 The Power of NP-Completeness	3
1.4 The Universal Tool: From Formal Proofs to Graph Coloring	3
2 Software Implementation	3
2.1 The Protocol: ZKP for Graph 3-Coloring	4
2.2 System Architecture: Client-Server Model	4
2.3 Key Technical Challenges & Solutions	5
3 Conceptual Exploration & Other Applications	5
3.1 ZKP for Sudoku	5
3.2 Investigation into Electronic Voting	5
4 Conclusion	6

Abstract

This project provides a comprehensive exploration of Zero-Knowledge Proofs (ZKPs), tracing their progression from foundational theory to practical software implementation and novel application. The study begins with a theoretical analysis of ZKP principles, based on Oded Goldreich's tutorial, to understand their core properties of completeness, soundness, and zero-knowledge. It then investigates the profound claim, originating from the work of Goldreich, Micali, and Wigderson, that ZKPs can exist for all NP problems by "reducing" formal mathematical proofs into graph 3-coloring problems.

The project then translates this theory into practice by implementing an interactive ZKP system for the classic NP-complete problem of graph 3-coloring. This implementation, built in C++ as a client-server application, addresses key software challenges, including the use of cryptographic hash commitments (SHA-256), the generation of secure randomness (salts), and the management of network communication between a Prover and a Verifier using C++ Asio. The resulting software successfully demonstrates the "commit-challenge-reveal-verify" protocol, allowing a Prover to statistically prove possession of a secret (a valid 3-coloring) without revealing it.

Finally, the project investigates the application of these foundational principles to other classic problems like Sudoku and the complex domain of electronic voting. This work explores conceptual e-voting models, analyzing how ZKPs could theoretically allow a tallying authority to prove the final result is correct without revealing individual votes, thereby highlighting the significant challenges and future potential of ZKPs in ensuring election integrity.

1 Theoretical Foundations

The project began with a study of the foundational theory behind Zero-Knowledge Proofs to understand what they are and why they are so powerful.

1.1 Definition: Conviction without Knowledge

At its core, a Zero-Knowledge Proof (ZKP) is a protocol between two parties, a **Prover** and a **Verifier**. The Prover's goal is to convince the Verifier that a specific statement is true, while the Verifier's goal is to be skeptical.

Unlike a traditional mathematical proof, which shares the entire sequence of arguments, a ZKP is *interactive* and *probabilistic*. It is designed to be:

- **Convincing:** The Verifier becomes statistically certain that the Prover's statement is true.
- **Zero-Knowledge:** The Verifier learns *nothing* else besides the fact that the statement is true. They do not learn the secret information (the "witness") that makes the statement true.

1.2 The Three Core Properties

A protocol is considered a ZKP if it satisfies three properties:

1. **Completeness:** If the Prover's statement is *true* and both parties follow the protocol, the Verifier will always be convinced.
2. **Soundness:** If the Prover's statement is *false* (i.e., they are trying to cheat), the protocol guarantees that the Verifier will detect the lie with very high probability.

3. **Zero-Knowledge:** The protocol leaks no information about the Prover's secret. This is formally defined by the concept of a **Simulator**—a hypothetical algorithm that can produce a "fake" conversation log that is statistically identical to a real one, *without* ever having access to the secret. If such a simulator exists, it proves that the real conversation must be "zero-knowledge," as the Verifier could have just faked the entire interaction themselves.

1.3 The Power of NP-Completeness

ZKPs are most powerful when applied to the class of problems known as **NP (Nondeterministic Polynomial time)**. These problems are famously "**hard to solve, but easy to check.**"

- **Sudoku:** Finding the solution to a blank puzzle is very difficult. Verifying a completed grid is trivial (just check the rows, columns, and boxes).
- **Graph 3-Coloring:** Finding a valid 3-coloring for a complex graph is extremely hard. Verifying a colored graph is easy (just check every edge to see if the two connected nodes have different colors).

Because these problems are easy to check, a Verifier can quickly validate a small "piece" of a solution. This is the mechanism that our interactive proof exploits.

1.4 The Universal Tool: From Formal Proofs to Graph Coloring

A key insight from the theoretical study, highlighted by the seminal paper by Oded Goldreich, Silvio Micali, and Avi Wigderson, is that ZKPs can be created for *every* problem in NP. The project studied the central idea behind this claim, which connects formal mathematics to graph coloring.

1. **Formalizing Proofs:** Any traditional mathematical proof (e.g., Andrew Wiles's 300-page proof of Fermat's Last Theorem) is written for human understanding. To be proven in ZK, it must first be written *formally* in a language like **1st-order logic**. This would expand the proof to millions of pages of logical statements.
2. **The NP-Complete Reduction:** The GMW paper's "vastness" comes from the fact that *any* statement in NP (which includes all 1st-order logic proofs) can be "reduced" or converted into an equivalent **Graph 3-Coloring problem**.
3. **The "Humongous Graph":** This reduction would transform the 1-million-page formal logic proof into a single, unimaginably massive graph. Proving the original theorem would now be equivalent to proving that "this humongous graph is 3-colorable."

Therefore, by building a working ZKP for graph 3-coloring, this project implements a protocol that is, in principle, a universal tool. It demonstrates a method that could be scaled to prove *any* mathematical theorem in zero-knowledge.

2 Software Implementation

The primary goal of this project was to build a working, interactive ZKP system from scratch to demonstrate the theory in practice.

2.1 The Protocol: ZKP for Graph 3-Coloring

Our software implements the classic interactive ZKP for graph 3-coloring. The process is a loop that runs a large number of times to achieve statistical soundness.

One Round of the Protocol:

1. **Commit:** The Prover takes its secret 3-coloring (e.g., {Node 1: Red, ...}). It then randomly "permutes" this coloring (e.g., all Red $\rightarrow$ Blue, Blue $\rightarrow$ Green, Green $\rightarrow$ Red). It then generates a large, secure random string (a "salt") for each vertex. It computes the SHA-256 digest of (color + salt) for every vertex and sends the complete list of digests to the Verifier.
2. **Challenge:** The Verifier receives the list of digests. It now knows the Prover is "locked in" to a solution. The Verifier randomly selects one *edge* from the graph (e.g., (Node 5, Node 12)) and sends this challenge to the Prover.
3. **Reveal:** The Prover receives the challenge. It sends back the *plaintext* color and salt for *only* the two vertices of that edge (e.g., (Blue, salt_5) and (Green, salt_12)).
4. **Verify:** The Verifier performs two checks:
 - It re-computes the hash SHA-256("Blue" + salt_5) and checks if it matches the digest for Node 5 that it received in the "Commit" step. (It does the same for Node 12).
 - It checks that the two revealed colors are different (Blue $\neq$ Green).
5. **Repeat:** If both checks pass, the Verifier is satisfied *for this round*. They repeat the entire process.

A Prover who doesn't know the solution might be able to cheat for one edge, but the probability of them successfully fooling the Verifier in numerous trials in a row is statistically zero.

2.2 System Architecture: Client-Server Model

The protocol was implemented as two separate C++ programs that communicate over a network.

- **Verifier (Server):** verifier.cpp
 - This program listens on a specific network port.
 - It reads the graph structure (without a coloring) from `std::cin`.
 - It accepts a connection from the Prover.
 - It runs the "Challenge" and "Verify" parts of the protocol loop.
- **Prover (Client):** prover.cpp
 - This program knows the secret solution. It reads the graph structure *and* the 3-coloring from `std::cin`.
 - It connects to the Verifier's IP address and port.
 - It runs the "Commit" and "Reveal" parts of the protocol loop.

2.3 Key Technical Challenges & Solutions

Building this system required solving several low-level C++ challenges:

- **Cryptographic Commitments:** Implemented using the **OpenSSL** library. The **SHA256** function was used to create a 256-bit (32-byte) digest, which served as a secure, one-way commitment. The digest was converted to a hexadecimal string for transmission.
- **Networking:** Implemented using the **Asio** (**asio.hpp**) library. This provided a modern, cross-platform C++ interface for TCP/IP sockets, allowing the Prover and Verifier to establish a connection and send/receive data.
- **Data Serialization:** Sockets only send raw streams of bytes, not C++ objects. A custom serialization protocol was designed to deconstruct complex types, like a `std::vector<std::string>`, into a simple, transmittable format. This was achieved by sending the item count, followed by the length and data for each item sequentially. The receiver then reassembled this byte stream into the original C++ object.
- **Byte Order:** When sending 32-bit integers (like *count* and *length*), we encountered the "Endianness" problem (Host vs. Network Byte Order). To solve this, the `htonl()` function was used by the sender and `ntohl()` by the receiver to ensure byte order was consistent.
- **Secure Randomness:** To generate cryptographic salts and to permute colors, the C++ `<random>` library was used, specifically `std::random_device` to get a true random seed, `std::mt19937` as the random engine, and `std::shuffle` to perform the permutation.

3 Conceptual Exploration & Other Applications

After building the core software, these principles were applied to other problems.

3.1 ZKP for Sudoku

A conceptual protocol for Sudoku was designed. It follows the same "commit-challenge-reveal" pattern:

1. **Commit:** Prover generates a random permutation of the numbers 1-9. It applies this permutation to its secret solution, generates salts, and sends the hash of each of the 81 cells.
2. **Challenge:** The Verifier challenges the Prover with one *constraint set* (a randomly chosen row, column, or 3x3 box).
3. **Reveal:** The Prover reveals the 9 (permuted) numbers and their 9 salts for that set.
4. **Verify:** The Verifier checks the hashes and also checks that the 9 revealed numbers are a valid set (i.e., contain 1-9 with no duplicates).
5. **Repeat:** This is repeated until the Verifier is statistically convinced.

3.2 Investigation into Electronic Voting

The project concluded with an investigation into the e-voting problem. This problem presents a fundamental conflict between two non-negotiable requirements:

1. **Ballot Secrecy:** A voter's choice must be private.

2. **Public Verifiability:** The public must be able to audit the election and confirm the final tally is correct.

This section of the project was an *exploration* and did not result in a complete, working model. Several approaches were considered, such as a model where a voter's "box" on a ledger can only be read by the voter. However, this creates a trust issue with the tallying algorithm.

A promising conceptual approach that was investigated is to have the central tallying authority produce a ZKP. In this model:

1. All votes are submitted in an encrypted form.
2. The tallying authority decrypts them in private and computes the final result.
3. The authority then publishes the final result (e.g., "Candidate A: 5,100, Candidate B: 4,900") *along with* a ZKP.
4. This ZKP would prove the statement: "This final tally is the correct sum of all valid, encrypted ballots, and I followed the decryption protocol correctly."

This would, in theory, allow the public to verify the *count* without ever seeing the *individual votes*, solving the conflict. This remains a highly complex field of research but demonstrates the profound, real-world potential of ZKPs.

4 Conclusion

This project successfully traced the concept of Zero-Knowledge Proofs from deep cryptographic theory to a tangible, working C++ client-server application. The process required understanding abstract concepts, such as the GMW paper's claim that any formal proof can be reduced to an NP-complete problem, as well as solving practical engineering challenges in networking, cryptography, and C++ development. The final software for graph 3-coloring demonstrates a functional interactive ZKP, and the conceptual analysis of Sudoku and e-voting highlights the broad applicability of these principles. This project builds a strong foundation for understanding the more advanced, non-interactive ZKPs (like **zk-SNARKs**) that power modern technologies.